Django Channels: Beginner to Advanced Full Notes

1. What is Django Channels?

Django Channels হলো Django-এর একটি extension, যা WebSocket, real-time communication, background events এবং asynchronous communication support করে।

সাধারণ Django HTTP request-response পদ্ধতিতে কাজ করে। Django Channels ব্যবহার করে client ও server-এর মধ্যে persistent connection তৈরি করা যায়।

HTTP vs WebSocket

HTTP:

- Client request পাঠায়।
- Server response দেয়।
- সাধারণত প্রতিটি request আলাদা।

WebSocket:

- Client ও server-এর মধ্যে persistent connection থাকে।
- দুই দিক থেকেই message পাঠানো যায়।
- Real-time application তৈরিতে ব্যবহার হয়।

Django Channels কোথায় ব্যবহার হয়?

- Real-time chat application
- Live notifications
- Live dashboard
- Typing indicators
- Online user presence
- Live tracking
- Multiplayer games
- Real-time collaboration

---

2. Important Concepts

ASGI

ASGI-এর পূর্ণরূপ Asynchronous Server Gateway Interface। এটি asynchronous Python application চালানোর standard interface।

Django Channels সাধারণত `asgi.py` ব্যবহার করে।

WSGI vs ASGI

- WSGI: Traditional synchronous web applications।

- ASGI: Asynchronous communication এবং WebSocket support করে।

Consumer

Consumer হলো Django Channels-এর সেই component, যা WebSocket connection ও message handle করে।

Routing

Routing নির্ধারণ করে কোন WebSocket URL কোন consumer-এর কাছে যাবে।

Channel Layer

Channel layer বিভিন্ন consumer-এর মধ্যে message পাঠানোর ব্যবস্থা করে।

Redis

Redis একটি in-memory data store। Production environment-এ Django Channels-এর channel layer হিসেবে Redis বহুল ব্যবহৃত হয়।

Group

Group-এর সাহায্যে একাধিক connected client-কে একই message পাঠানো যায়।

---

3. Installation

প্রথমে virtual environment তৈরি করো।

```
python -m venv venv
```

Windows:

```
venv\Scripts\activate
```

Packages install করো:

```
pip install django channels channels-redis
```

Project তৈরি করো:

```
django-admin startproject config .
python manage.py startapp chat
```

Project structure:

```
project/
├── manage.py
├── config/
│   ├── __init__.py
│   ├── settings.py
│   ├── urls.py
```

```
│   ├── asgi.py
│   └── routing.py
└── chat/
    ├── __init__.py
    ├── consumers.py
    ├── routing.py
    ├── views.py
    └── urls.py
```

---

4. Configure settings.py

`config/settings.py` ফাইলে:

```
INSTALLED_APPS = [
    "daphne",
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",
    "channels",
    "chat",
]

ASGI_APPLICATION = "config.asgi.application"
```

যদি `daphne` ব্যবহার করতে চাও, তাহলে install করো:

```
pip install daphne
```

Production বা multi-process application-এর জন্য Redis channel layer configure করো:

```
CHANNEL_LAYERS = {
    "default": {
        "BACKEND": "channels_redis.core.RedisChannelLayer",
        "CONFIG": {
            "hosts": [("127.0.0.1", 6379)],
        },
    },
}
```

এই configuration-এর জন্য Redis server চালু থাকতে হবে।

---

5. Create Your First Consumer

`chat/consumers.py`:

```
from channels.generic.websocket import AsyncWebsocketConsumer


class EchoConsumer(AsyncWebsocketConsumer):

    async def connect(self):
        await self.accept()
        await self.send(text_data="WebSocket Connected!")

    async def disconnect(self, close_code):
        print("WebSocket disconnected:", close_code)

    async def receive(self, text_data=None, bytes_data=None):
        if text_data is not None:
            await self.send(text_data=f"You sent: {text_data}")
```

এখানে:

- `connect()` — client connect করলে চলে।
- `accept()` — WebSocket connection গ্রহণ করে।
- `receive()` — client message পাঠালে চলে।
- `send()` — client-কে message পাঠায়।
- `disconnect()` — connection বন্ধ হলে চলে।

---

6. WebSocket Routing

`chat/routing.py`:

```
from django.urls import path
from .consumers import EchoConsumer

websocket_urlpatterns = [
    path("ws/echo/", EchoConsumer.as_asgi()),
]
```

`config/routing.py`:

```
from chat.routing import websocket_urlpatterns

__all__ = ["websocket_urlpatterns"]
```

`config/asgi.py`:

```
import os

from django.core.asgi import get_asgi_application
from channels.routing import ProtocolTypeRouter, URLRouter
from channels.auth import AuthMiddlewareStack
from django.urls import path

os.environ.setdefault("DJANGO_SETTINGS_MODULE", "config.settings")

django_asgi_app = get_asgi_application()

from chat.routing import websocket_urlpatterns

application = ProtocolTypeRouter({
    "http": django_asgi_app,
    "websocket": AuthMiddlewareStack(
        URLRouter(websocket_urlpatterns)
    ),
})
```

`get_asgi_application()` আগে initialize করা হয়েছে, যাতে Django app setup সম্পন্ন হওয়ার পর consumer ও model imports সঠিকভাবে কাজ করতে পারে।

---

7. Create a Test HTML Page

`chat/views.py`:

```
from django.shortcuts import render


def home(request):
    return render(request, "chat/index.html")
```

`chat/urls.py`:

```
from django.urls import path
from . import views

urlpatterns = [
    path("", views.home, name="home"),
]
```

`config/urls.py`:

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path("admin/", admin.site.urls),
    path("", include("chat.urls")),
]
```

HTML file তৈরি করো:

`chat/templates/chat/index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Django Channels Echo</title>
</head>
<body>
    <h1>WebSocket Echo Test</h1>

    <input id="messageInput" type="text">
    <button onclick="sendMessage()">Send</button>

    <ul id="messages"></ul>

    <script>
        const scheme = location.protocol === "https:" ? "wss" : "ws";

        const socket = new WebSocket(
            `${scheme}://${location.host}/ws/echo/`
        );

        socket.onopen = function () {
            console.log("Connected");
        };

        socket.onmessage = function (event) {
            const li = document.createElement("li");
            li.textContent = event.data;
            document.getElementById("messages").appendChild(li);
        };

        socket.onclose = function () {
```

```
            console.log("Disconnected");
        };

        socket.onerror = function (error) {
            console.error("WebSocket error:", error);
        };

        function sendMessage() {
            const input = document.getElementById("messageInput");

            if (socket.readyState === WebSocket.OPEN) {
                socket.send(input.value);
                input.value = "";
            }
        }
    </script>
</body>
</html>
```

Run:

```
python manage.py migrate
python manage.py runserver
```

Browser-এ যাও:

```
http://127.0.0.1:8000/
```

---

8. Sync vs Async Consumers

Synchronous Consumer

```
from channels.generic.websocket import WebsocketConsumer


class MyConsumer(WebsocketConsumer):

    def connect(self):
        self.accept()

    def receive(self, text_data=None, bytes_data=None):
        self.send(text_data="Hello from sync consumer!")
```

Asynchronous Consumer

```
from channels.generic.websocket import AsyncWebsocketConsumer


class MyAsyncConsumer(AsyncWebsocketConsumer):

    async def connect(self):
        await self.accept()

    async def receive(self, text_data=None, bytes_data=None):
        await self.send(text_data="Hello from async consumer!")
```

পার্থক্য:

- Sync consumer সাধারণ synchronous code-এর জন্য।
- Async consumer asynchronous operations-এর জন্য।
- Async consumer-এর ভিতরে synchronous database operations সরাসরি চালালে event loop block হতে পারে।
- Async consumer-এ asynchronous ORM methods অথবা `database_sync_to_async` ব্যবহার করো।

---

9. JSON Messages

Client ও server-এর মধ্যে structured data পাঠানোর জন্য JSON ব্যবহার করা যায়।

```
import json
from channels.generic.websocket import AsyncWebsocketConsumer


class JsonConsumer(AsyncWebsocketConsumer):

    async def connect(self):
        await self.accept()

    async def receive(self, text_data=None, bytes_data=None):
        try:
            data = json.loads(text_data or "{}")
        except json.JSONDecodeError:
            await self.send(text_data=json.dumps({
                "error": "Invalid JSON"
            }))
            return

        message = data.get("message", "")
```

```
        await self.send(text_data=json.dumps({
            "message": message,
            "status": "success"
        }))
```

Client থেকে:

```
socket.send(JSON.stringify({
    message: "Hello Django"
}));
```

---

10. Channel Layer and Redis

একাধিক client-এর মধ্যে message broadcast করতে channel layer ব্যবহার করা হয়।

In-memory channel layer development ও testing-এ কাজে লাগে, কিন্তু এটি production-এর multi-process setup-এর জন্য উপযুক্ত নয়।

Redis install:

```
pip install channels-redis
```

Redis server চালু করো এবং `settings.py` -এ configuration যোগ করো:

```
CHANNEL_LAYERS = {
    "default": {
        "BACKEND": "channels_redis.core.RedisChannelLayer",
        "CONFIG": {
            "hosts": [("127.0.0.1", 6379)],
        },
    },
}
```

Redis channel layer একাধিক application process-এর মধ্যে event পাঠাতে সাহায্য করে।

---

11. Complete Group Chat Consumer

`chat/consumers.py`:

```
import json
from channels.generic.websocket import AsyncWebsocketConsumer


class ChatConsumer(AsyncWebsocketConsumer):

    async def connect(self):
```

```
        self.room_name = self.scope["url_route"]["kwargs"]["room_name"]
        self.room_group_name = f"chat_{self.room_name}"

        await self.channel_layer.group_add(
            self.room_group_name,
            self.channel_name
        )

        await self.accept()

    async def disconnect(self, close_code):
        await self.channel_layer.group_discard(
            self.room_group_name,
            self.channel_name
        )

    async def receive(self, text_data=None, bytes_data=None):
        try:
            data = json.loads(text_data or "{}")
        except json.JSONDecodeError:
            return

        message = data.get("message", "")

        if not isinstance(message, str) or not message.strip():
            return

        await self.channel_layer.group_send(
            self.room_group_name,
            {
                "type": "chat_message",
                "message": message.strip(),
            }
        )

    async def chat_message(self, event):
        await self.send(text_data=json.dumps({
            "message": event["message"]
        }))
```

Room Routing

`chat/routing.py`:

```
from django.urls import path
from .consumers import ChatConsumer

websocket_urlpatterns = [
    path(
        "ws/chat/<str:room_name>/",
        ChatConsumer.as_asgi(),
    ),
]
```

Group Methods

- `group_add()` — consumer-কে group-এ যুক্ত করে।
- `group_discard()` — consumer-কে group থেকে সরায়।
- `group_send()` — group-এর সবাইকে event পাঠায়।
- `chat_message()` — `type="chat_message"` event handle করে।

গুরুত্বপূর্ণ: এই example-টি basic demonstration। এখানে authentication, room permission, rate limiting এবং message persistence যোগ করা হয়নি।

---

12. WebSocket Client for Chat

HTML-এর JavaScript অংশ:

```
<input id="messageInput" type="text">
<button onclick="sendMessage()">Send</button>
<ul id="messages"></ul>

<script>
    const scheme = location.protocol === "https:" ? "wss" : "ws";

    const roomName = "general";

    const socket = new WebSocket(
        `${scheme}://${location.host}/ws/chat/${roomName}/`
    );

    socket.onmessage = function (event) {
        const data = JSON.parse(event.data);
        const li = document.createElement("li");

        li.textContent = data.message;

        document.getElementById("messages").appendChild(li);
```

```
    };

    function sendMessage() {
        const input = document.getElementById("messageInput");

        if (socket.readyState === WebSocket.OPEN) {
            socket.send(JSON.stringify({
                message: input.value
            }));

            input.value = "";
        }
    }
</script>
```

একই room name দিয়ে একাধিক browser tab খুলে test করো। একটি tab থেকে message পাঠালে অন্য tab-গুলোও message পাবে।

---

13. Scope and Authentication

`self.scope`-এ connection সম্পর্কিত তথ্য থাকে।

উদাহরণ:

```
user = self.scope["user"]

room_name = self.scope["url_route"]["kwargs"]["room_name"]
```

`AuthMiddlewareStack` Django session-based authentication-এর মাধ্যমে WebSocket connection-এ user information উপলব্ধ করতে পারে।

কিন্তু **authentication এবং authorization এক জিনিস নয়**:

- Authentication: user কে?
- Authorization: user এই room-এ প্রবেশ করতে পারবে কি?

Protected chat room-এর ক্ষেত্রে `connect()`-এ permission যাচাই করো:

```
from channels.generic.websocket import AsyncWebsocketConsumer


class PrivateChatConsumer(AsyncWebsocketConsumer):

    async def connect(self):
        user = self.scope["user"]

        if not user.is_authenticated:
```

```
            await self.close()
            return

        # এখানে room membership ও permission যাচাই করতে হবে।
        await self.accept()
```

এটি শুধু authentication check-এর উদাহরণ। বাস্তব private room-এর জন্য membership authorization-ও implement করতে হবে।

---

14. Database Operations in Async Consumers

Async consumer-এ synchronous ORM call সরাসরি ব্যবহার করা এড়িয়ে চলো।

`chat/models.py`:

```
from django.conf import settings
from django.db import models


class Room(models.Model):
    name = models.CharField(max_length=100, unique=True)

    def __str__(self):
        return self.name


class Message(models.Model):
    room = models.ForeignKey(
        Room,
        on_delete=models.CASCADE,
        related_name="messages",
    )

    user = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE,
    )

    content = models.TextField()
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f"{self.user} - {self.room}"
```

Migration:

```
python manage.py makemigrations
python manage.py migrate
```

Database Helper

```
from channels.db import database_sync_to_async
from .models import Message, Room


@database_sync_to_async
def save_message(room_name, user, content):
    room = Room.objects.get(name=room_name)

    return Message.objects.create(
        room=room,
        user=user,
        content=content,
    )


@database_sync_to_async
def get_recent_messages(room_name):
    room = Room.objects.get(name=room_name)

    return list(
        Message.objects.filter(room=room)
        .select_related("user")
        .order_by("-created_at")[:50]
    )
```

Consumer থেকে ব্যবহার:

```
message = await save_message(
    self.room_name,
    self.scope["user"],
    "Hello!",
)
```

Database access করার আগে user authenticated কি না এবং room-এ তার permission আছে কি না যাচাই করতে হবে। `Room.DoesNotExist`-এর মতো database errors-ও handle করো।

---

15. Notifications

Notification event পাঠানোর ধারণা:

```
await self.channel_layer.group_send(
    f"user_{user_id}",
    {
        "type": "notification_message",
        "message": "You have a new notification",
    }
)
```

Consumer event handler:

```
async def notification_message(self, event):
    await self.send(text_data=json.dumps({
        "type": "notification",
        "message": event["message"],
    }))
```

Notification group-এ user-কে যুক্ত করার আগে authentication ও ownership যাচাই করতে হবে। অন্য user-এর notification group-এ unauthorized access দেওয়া যাবে না।

---

16. Typing Indicator

Client থেকে typing event:

```
socket.send(JSON.stringify({
    type: "typing",
    is_typing: true
}));
```

Server-এ event type অনুযায়ী আলাদা handling যোগ করো:

```
if data.get("type") == "typing":
    await self.channel_layer.group_send(
        self.room_group_name,
        {
            "type": "typing_event",
            "is_typing": bool(data.get("is_typing")),
        }
    )
    return
```

Event handler:

```
async def typing_event(self, event):
    await self.send(text_data=json.dumps({
```

```
            "type": "typing",
            "is_typing": event["is_typing"],
        }))
```

বাস্তব application-এ typing event-এর সঙ্গে sender identity যোগ করো এবং event spam সীমিত করো।

---

17. Online Presence

Online presence তৈরি করতে সাধারণত:

1. User connect করলে presence record update করা হয়।
2. Disconnect হলে record update করা হয়।
3. Multiple browser tabs বা devices থাকলে connection count বিবেচনা করা হয়।
4. Unexpected disconnection-এর জন্য expiry বা heartbeat ব্যবস্থা রাখা হয়।

শুধু `connect()` এবং `disconnect()` দিয়ে online status নির্ভুল রাখা যায় না, কারণ network failure হলে disconnect event সব সময় প্রত্যাশিতভাবে নাও আসতে পারে।

---

18. WebSocket Security

Production application-এ:

- WebSocket URL-এ `wss://` ব্যবহার করো।
- `AllowedHostsOriginValidator` বা উপযুক্ত `OriginValidator` ব্যবহার করো।
- Authentication যাচাই করো।
- Room membership ও permissions যাচাই করো।
- Message size limit রাখো।
- Rate limiting implement করো।
- Client input validate করো।
- Sensitive data log করা এড়িয়ে চলো।
- User-supplied content HTML হিসেবে render কোরো না।
- Redis-কে public internet-এ উন্মুক্ত রেখো না।

Origin Validation Example

`config/asgi.py`:

```
from channels.security.websocket import AllowedHostsOriginValidator
from channels.auth import AuthMiddlewareStack
from channels.routing import ProtocolTypeRouter, URLRouter
from django.core.asgi import get_asgi_application
import os

os.environ.setdefault("DJANGO_SETTINGS_MODULE", "config.settings")
```

```
django_asgi_app = get_asgi_application()

from chat.routing import websocket_urlpatterns

application = ProtocolTypeRouter({
    "http": django_asgi_app,
    "websocket": AllowedHostsOriginValidator(
        AuthMiddlewareStack(
            URLRouter(websocket_urlpatterns)
        )
    ),
})
```

Production-এ `ALLOWED_HOSTS`-এ সঠিক domain configure করো। Origin validation authorization-এর বিকল্প নয়।

---

19. Testing WebSocket Consumers

Install:

```
pip install channels
```

`chat/tests.py`:

```
from channels.testing import WebsocketCommunicator
from django.test import TransactionTestCase

from config.asgi import application


class EchoConsumerTests(TransactionTestCase):

    async def test_echo_connection(self):
        communicator = WebsocketCommunicator(
            application,
            "/ws/echo/",
        )

        connected, _ = await communicator.connect()
        self.assertTrue(connected)

        response = await communicator.receive_from()
        self.assertEqual(response, "WebSocket Connected!")
```

```
        await communicator.send_to("Hello")
        response = await communicator.receive_from()

        self.assertEqual(response, "You sent: Hello")

        await communicator.disconnect()
```

Run:

```
python manage.py test
```

Project-এর routing অনুযায়ী test URL পরিবর্তন করো।

---

20. Common Errors

Error: No route found

সম্ভাব্য কারণ:

- WebSocket URL ভুল।
- `websocket_urlpatterns` configure করা হয়নি।
- `URLRouter` সঠিকভাবে setup করা হয়নি।

Error: Cannot connect to Redis

সম্ভাব্য কারণ:

- Redis server বন্ধ।
- Host বা port ভুল।
- Redis connection configuration ভুল।

Error: SynchronousOnlyOperation

সম্ভাব্য কারণ:

- Async context-এ synchronous ORM ব্যবহার করা হয়েছে।

সমাধান:

- Async ORM methods ব্যবহার করো।
- অথবা `database_sync_to_async` ব্যবহার করো।

Error: WebSocket closes immediately

সম্ভাব্য কারণ:

- Consumer-এ exception।
- Authentication বা Origin validation failure।
- Routing ভুল।
- Connection accept করা হয়নি।

Error: `group_send` কাজ করছে না

সম্ভাব্য কারণ:

- Redis/channel layer configuration ভুল।
- Consumer group-এ যোগ হয়নি।
- Event-এর `type` ও handler name মেলেনি।

---

21. Production Deployment

Production deployment-এর আগে:

1. `DEBUG = False` রাখো।
2. `ALLOWED_HOSTS` configure করো।
3. Environment variables-এ secrets রাখো।
4. Redis channel layer configure করো।
5. ASGI-capable server ব্যবহার করো।
6. HTTPS/WSS এবং reverse proxy configure করো।
7. Logging ও monitoring setup করো।
8. Database connection এবং performance test করো।
9. WebSocket authentication ও authorization test করো।
10. Redis-এর security ও availability নিশ্চিত করো।

Development server:

```
python manage.py runserver
```

এটি development-এর জন্য। Production deployment-এ উপযুক্ত ASGI server ও infrastructure configure করতে হবে।

---

22. Interview Questions

Q1. What is Django Channels?

Django Channels Django-তে WebSocket এবং asynchronous protocols support করার extension।

Q2. What is ASGI?

ASGI হলো asynchronous Python web application-এর standard interface।

Q3. What is a Consumer?

Consumer WebSocket connection, incoming message এবং outgoing event handle করে।

Q4. What is a Channel Layer?

Channel layer application-এর বিভিন্ন consumer-এর মধ্যে event/message আদান-প্রদান করতে সাহায্য

করে।

Q5. Why Redis?

Redis multiple workers/process-এর মধ্যে channel layer messages আদান-প্রদানে ব্যবহৃত হয়।

Q6. What is a Group?

Group হলো একাধিক channel-এর collection, যেখানে broadcast event পাঠানো যায়।

Q7. What is `group_send()`?

এটি নির্দিষ্ট group-এর channel-গুলোতে event পাঠায়।

Q8. What is `scope`?

Scope connection-সম্পর্কিত তথ্য ধারণ করে, যেমন user ও URL parameters।

Q9. What is the difference between authentication and authorization?

Authentication user-এর পরিচয় যাচাই করে; authorization user-এর access permission যাচাই করে।

Q10. How do you access the database in an async consumer?

Async ORM methods অথবা `database_sync_to_async` ব্যবহার করে।

Q11. Why use WebSockets?

Persistent, bidirectional communication ও low-latency updates-এর জন্য।

Q12. Is the in-memory channel layer suitable for production?

সাধারণ multi-process production deployment-এর জন্য নয়; Redis-এর মতো shared channel layer ব্যবহার করা ভালো।

---

23. Practice Projects

Beginner:

- Echo WebSocket
- Live notification demo

Intermediate:

- Group chat
- Typing indicator
- Online user presence

Advanced:

- Private messaging
- Persistent chat with database
- Role-based private rooms
- Live dashboard
- Reconnect handling

- Rate-limited real-time notification system

প্রতিটি project-এ authentication, authorization, input validation, testing ও error handling যোগ করার চেষ্টা করো।

---

24. 12-Week Learning Roadmap

Week 1–2: Python async/await, Django fundamentals, HTTP ও ASGI।

Week 3–4: WebSocket basics, Consumers, Routing, Scope।

Week 5–6: Channel Layer, Redis, Groups ও Chat Application।

Week 7–8: Authentication, Authorization, Database integration।

Week 9–10: Notifications, Typing Indicator, Presence ও Testing।

Week 11: Security, Performance ও Error Handling।

Week 12: Production deployment ও final project।

---

25. Official Documentation

- Django: https://docs.djangoproject.com/
- Django Channels: https://channels.readthedocs.io/
- Channels Tutorial: https://channels.readthedocs.io/en/stable/tutorial/index.html
- Channels Deployment: https://channels.readthedocs.io/en/stable/deploying.html
- Redis: https://redis.io/docs/

Final Advice

প্রথমে EchoConsumer তৈরি করে WebSocket connection বুঝো। এরপর Redis ও Group Chat implement করো। তারপর authentication, database persistence, private rooms, automated tests এবং production deployment শিখো।

এভাবে ধাপে ধাপে practice করলে Django Channels beginner থেকে advanced level পর্যন্ত শেখা সহজ হবে।